

Test Plan — Improving Supply Chain Oregon

David Foster, Luke Garci, Caden Smith | May 14th, 2026

Project Overview

Improving Supply Chain Oregon is a multi-tenant Flask/MySQL web application that helps Oregon agricultural stakeholders' model and analyze delivery scenarios across the middle stages of the food supply chain: storage, aggregation, processing, and distribution. Users define locations (Farms, Hubs, Stores), create delivery routes, assign vehicles and drivers, build product manifests, and receive full cost breakdowns covering fuel, depreciation, insurance, maintenance, and driver wages. Trip distances are retrieved via the Mapbox API. Authentication uses JWT cookies with mandatory TOTP two-factor authentication.

Testing Needs & Areas of Concern

Financial Calculation Engine

The core value of the app is its cost-calculation logic (`logic.py`, `depreciation_insurance.py`). Errors directly mislead users about route profitability. Key concerns:

- Depreciation uses $(\text{Purchase} - \text{Salvage}) / (\text{Annual Miles} \times 8 \text{ yrs}) \times \text{Trip Miles}$. All inputs can be zero or null, requiring safety-division handling.
- Insurance and maintenance follow the same per-mile formula and must be verified independently.
- Driver cost depends on separate drive and load rates and times from different DB snapshot columns.
- Fuel cost has two paths: Mapbox estimate or $\text{MPG} \times \text{gas price}$. Both must be tested.
- Profit and margin are computed from both entered revenue and manifest-calculated revenue separately.
- All helpers (`safe_float`, `safe_int`, `to_decimal`) must handle None, empty string, and non-numeric input without raising exceptions.

Mapbox API Integration

The app calls Mapbox synchronously on every cost calculation. If the API fails or returns no result, the app silently defaults to 0 miles and 0 minutes, making all distance-based costs zero with no error shown to the user. Timeout handling and address-string construction (concatenated from DB fields) also need verification.

Multi-Tenant Data Isolation

Every table uses a composite (tenant_id, record_id) primary key. All DB access functions filter by tenant_id from the JWT. A failure here could expose one organization's data to another. Scoped reads, writes, and deletes must never cross tenant boundaries, and an altered tenant_id in a JWT must not grant access to foreign records.

Authentication & Session Security

JWT access tokens (HttpOnly cookies) plus TOTP 2FA protect all routes. Concerns: expired/tampered tokens must redirect cleanly; users without confirmed TOTP must be redirected to set up on every protected route; anonymous recovery cookies must not allow privilege escalation; password reset tokens must be single-use and expiring.

Form Validation

numeric parse helpers must handle edge cases like 'inf', '-0', and comma-formatted numbers without crashing.

Testing Methods

White-Box Unit Testing (pytest)

logic.py and depreciation_insurance.py are pure functions with no external dependencies, making them ideal for exhaustive unit testing

Function	Key Test Cases
trip_depreciation_cost()	Normal; salvage > purchase (floor at 0); mileage = 0; all None
trip_insurance_cost()	Normal; mileage = 0; trip_miles = 0
calculate_driver_costs()	Zero times; fractional hours; rate = 0
calculate_fuel_cost()	estimated_cost > 0 bypasses MPG; mpg = 0; all zero
safe_float / safe_int	None, "", 'abc', '1,000', 'inf', '-0'
aggregate_manifest_totals()	Empty list; None prices; zero quantity

White-Box Integration Testing (pytest + test DB)

The data-access layer requires integration tests against a dedicated test database. Priority cases:

- Create a route under Tenant A, query with Tenant B credentials — assert empty result.
- Load dashboard with 10 routes — assert DB query count is bounded, not N+1.

Black-Box / Boundary Testing

Endpoints and forms are tested as opaque units, verifying that rules are enforced regardless of implementation:

- POST /routes/new with missing fields — assert 400 and modal stays open with data preserved.
- POST /routes/new with identical origin and destination — assert validation error.
- POST /routes/<id>/load/add with quantity = 0 or non-existent product_id — assert graceful rejection.

- All protected endpoints without a valid JWT — assert redirect to /auth/login.
- JWT with altered tenant_id — assert no foreign data returned.
- GET /routes/<nonexistent>/view — assert 404.

Mocking External Services

Mapbox calls are mocked with unittest.mock.patch so tests run without network access. Three scenarios: normal return (25.0 miles, 45.0 min); return (None, None) to verify silent-zero fallback; raise requests.Timeout to verify exception handling.

Security Testing

Manual security tests using JWT decoding tools (jwt.io):

- Tampered JWT (wrong secret) assert logout redirect.
- Expired access token — assert logout redirect.
- Anonymous session accessing TOTP-required routes — assert blocked.
- IDOR: access /routes/<id>/view with another tenant's route_id — assert 404.
- totp_confirmed = False user accessing protected routes — assert redirect to TOTP setup.

User Acceptance Testing (UAT)

Acceptance testing will be done 5/19 or 5/26 (dependent on stakeholder schedule) with Jim Cupples and an external stakeholder: creating a full route from scratch; adding manifest items and verifying cost totals against hand calculations; editing a route and confirming recalculation; exporting a CSV and opening in Excel; attempting to delete a location in use and receiving a friendly error. Feedback is captured via think-aloud notes.

Kanban board

Trello Link: <https://trello.com/b/74R4l40b/using-ai-to-improve-supply-chains-in-oregon>

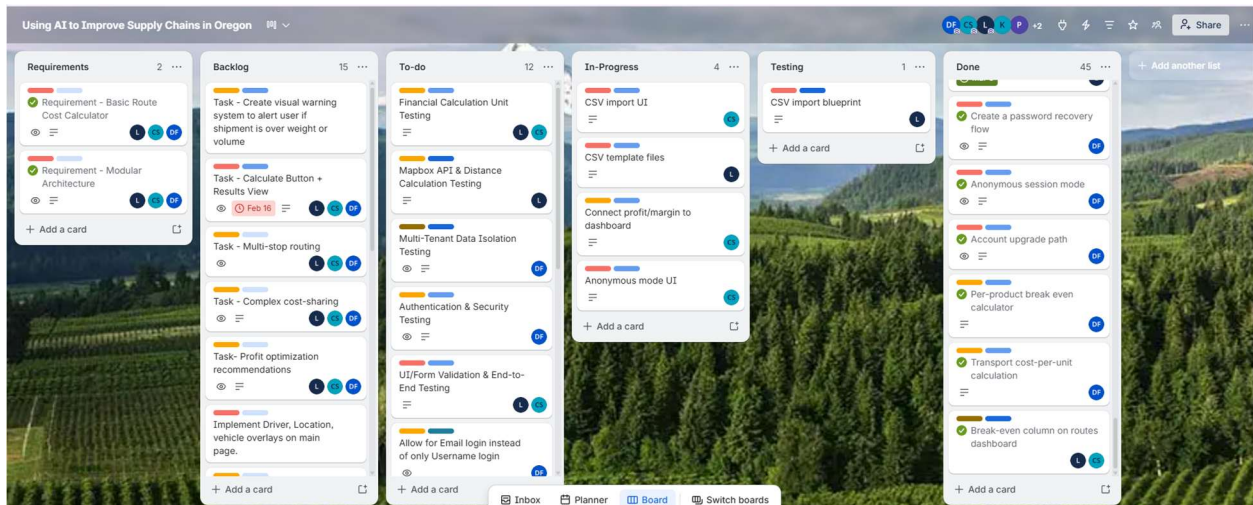


Figure 1: Updated board with testing tasks